GRAFT-Game-theoritic Reinforcement learning for Adaptive Fraud detection and Trust aware routing

Anurag Marda[1],Supraja Soudu[2], Anshul Agarwal[3]

[1] Computer Science and Engineering, Visvesvaraya National Institute of Technology (VNIT) Nagpur, Nagpur, India

Reception date of the manuscript: dd/mm/yyyy
Acceptance date of the manuscript: dd/mm/yyyy
Publication date: dd/mm/yyyy

Abstract

Decentralised peer-to-peer networks must address two tightly coupled security problems: selecting routing paths that avoid untrustworthy intermediate nodes and identifying fraudulent destination nodes in real time. Most existing methods handle routing security and fraud detection separately, which limits their adaptability in dynamic trust environments. The core challenge is that routing decisions and fraud judgements cannot be made independently, because a node that appears trustworthy from one direction may be flagged as fraudulent from another, making joint real-time inference across the full graph non-trivial. This paper presents GRAFT a novel game-theoretic reinforcement learning framework that jointly performs safe routing and real-time fraud detection. Each routing hop is modelled as a non-cooperative repeated Prisoner's Dilemma in which the normalised trust score defines the cooperation payoff, and the routing agent seeks to maximise cumulative path utility using persistent Q-learning with explicit train-test phase separation, allowing routing knowledge to be retained across sessions. The learned Q-table converges to a Nash-equilibrium routing policy, integrated with a four-signal ensemble classifier-based on average incoming trust, explicit fraud vote ratio, negative vote ratio, and outgoing behavioural score-alongside a live reinforcement-learning reputation model that updates during each routing episode. Experimental evidence has demonstrated an aggregate of 100% success in transaction routing throughout the Bitcoin OTC network's entire deployment and detected 99.6% accuracy of fraud at five thousand eight hundred fifty-eight classifiable nodes, without any missed by at least one malicious node (i.e., 100% recall) and the permanent blacklisting of all known prior from their first reported to prevent any further re-training and subsequent supervision labels being required following their first appearance.

Keywords- Reinforcement Learning; Trust-Based Routing;Fraud De tection; Bitcoin OTC; Game Theory; Prisoner's Dilemma; Nash Equilibrium; Node Reputation; Peer-to-Peer Networks

I. INTRODUCTION

Decentralized peer-to-peer networks let users interact directly no central authority calling the shots. That's great for openness and autonomy, but it also makes things messy when it comes to security. Take trust-based networks like the Bitcoin OTC trust graph. When two people want to connect, their messages usually hop through a bunch of other users -most of them total strangers, with unknown intentions. Now, the sender isn't just hoping those go-betweens are honest. They also have to trust the person they're trying to reach isn't a scammer.

Existing approaches have attempted to address these challenges; however, the proposed solutions typically address only partial aspects of the problem. Some research efforts concentrate on identifying reliable routing mechanisms, while others focus on detecting fraudulent activities at individual checkpoints. Notably, very few methodologies successfully integrate both components simultaneously, resulting in a significant gap in the literature: the absence of a comprehensive framework for ensuring end-to-end security.

Classic routing protocols like AODV and OLSR don't help much-they choose paths using basic measures like hop count or link quality but don't care if a node is actually trustworthy. Fraud detection tools, on the other hand, usually lean on machine learning and sift through old data, flagging suspicious users - but only at the destination and without influencing the route taken. The upshot? None of these systems dodge evil intermediaries while catching scams at the end.

This paper tackles both problems together. It introduces a single framework that blends trust-aware routing with on-the-fly fraud detection, using reinforcement learning as the glue. Each step in the route acts out a repeat of the Prisoner's Dilemma, with trust scores shaping rewards for cooperation or penalties for betrayal. There's a Q-learning agent that picks up on sneaky behavior over time, constantly improves its routing choices, and blacklists bad actors for good. Meanwhile, an ensemble classifier-one that takes in four different signals and feeds off these dynamic trust updates-screens each destination in real time, adapting as it goes. The result: a network that learns from experience, routes smartly, and keeps scammers at bay, end to end.

This paper makes the following main contributions:

1. GRAFT unified novel game-theoretic routing framework that models each hop as a Prisoner's Dilemma with normalised trust weights, deriving a Nash-equilibrium routing policy via Q-learning - the first such formulation to jointly encode cooperation incentives and malicious-edge penalties directly into the reward structure.

2. A persistent Q-learning architecture with train–deployment phase separation that retains routing knowledge across sessions via a saved Q-table, enabling deterministic exploitation at deployment without retraining or labelled supervision.

3. A real-time four-signal ensemble fraud classifier that integrates incoming trust, fraud reports, negative feedback,

and RL-derived reputation scores, dynamically blacklisting malicious nodes mid-route without requiring model restarts.

4. Exhaustive evaluation on the Bitcoin OTC signed trust network (5,881 nodes, 35,592 edges) demonstrates 100% routing success rate at deployment, 99.6% fraud detection accuracy, and perfect recall across 5,858 classifiable nodes - outperforming existing behavioural and ML-based baselines in both accuracy and network coverage.

II. RELATED WORK

Research in this area covers a lot of ground-people have looked at supervised fraud detection, unsupervised graph mining, user behavior analysis, and trust-based cluster- ing on Bitcoin networks. Sure, these studies offer some sharp insights, but they usually treat fraud detection, anomaly spotting, and identity analysis as separate problems. No one really brings adaptive fraud detection and safe path discovery together under one routing-aware system.

2.1 Ensemble and Classical Machine Learning Approaches: Lately, a lot of research has been about catching fraud in blockchain data using both ensemble and classic supervised learning models. Take Nayyer[1] and his team-they mixed Decision Trees, Naive Bayes, and KNN, then used Random Forest as a meta-learner. To help balance things out, they brought in ADASYN. This combo delivered some pretty strong classification results on the Bitcoin Heist dataset. Singh's[2] group tested out K-means, Isolation Forest, and SVMs, adding a layer of feature selection to improve things. Ashfaq's[3] team took a different route: they merged XGBoost and Random Forest in a smart contract system that checks for fraud as it happens, right on the blockchain. On the Ethereum side, Pahuja and Kamal[4] built the EnLFADE framework, matching LightGBM with feature selection to sniff out fraud. But here's the catch-all these systems work as static, batch classifiers. They train on historical, labeled data and just don't adapt as new stuff rolls in. On top of that, none of them handle trust-aware routing, so there's definitely room to grow.

2.2 Unsupervised Clustering and Anomaly Detection Methods: Researchers often turn to unsupervised learning and clustering to spot suspicious activity in transaction data and network graphs. For example, Zambre and Shah[5] use k-means clustering on how quickly accounts move money to flag possible theft. Pham and Lee[6] take a slightly broader approach, applying k-means, Mahalanobis distance, and one-class SVM to both user and transaction graphs to catch unusual behavior. Wang[7] and his team cluster trust-related behaviors on the Bitcoin OTC platform to highlight outliers. Then there's Nan and Tao[8], whose BGID framework links together autoencoders, k-means, and local outlier probabilities to track down intermediaries working with mixing services. These methods work well for sniffing out structural anomalies. But they have some real gaps: they run in batches, don't adapt to changes over time, and don't offer solutions for safe routing or ongoing, continuous learning.

2.3 Graph Neural Networks and Deep Learning Models: Researchers have really dug into deep learning for spotting fraud on blockchain networks. Adloori[9] and his team rolled out GAT-ResNet using the Elliptic dataset. Xue's[10] group came up with TAS-GNN for signed graph learning on Bitcoin-Alpha. Then you've got Asiri and Somasundaram[11], who boosted GCN models by working graph centrality into the mix-to pick out illicit transactions more effectively. Kamisetty[12] and colleagues took a broad look, surveying everything from ANN and RNN to CNN and autoencoders for Bitcoin fraud. The thing is, these models handle node-level classification well since they use the graph's structure. But they're stuck as static predictors-they don't adapt on the fly or offer any routing intelligence.

2.4 Behavioral and Identity-Based Detection Methods: Some research looks more at behavioral profiling and figuring out who's behind certain actions than at building network trust. Monaco[13], for example, digs into long-term transaction timing and uses a kNN-style classifier to pick out users. Another study by Viswam and Darsan[14] takes social media posts, pulls out linguistic features, and runs a Multinomial Naive Bayes model to spot deceptive Bitcoin-related content. The catch with these methods? They lean hard on manually chosen behavioral features. They aren't built to work with trust graphs, and they don't help much when you need to choose a safe path through networks where adversaries might be lurking.

Most current methods treat fraud detection and routing like they're totally separate. They usually rely on static or batch learning, so they have a hard time keeping up with new types of fraud. Plus, they don't tie trust-aware path selection together with real-time fraud checks-there's no single, learning-driven system that does it all at once.The framework developed in this paper fills that gap. It blends persistent reinforcement learning for safe routing with real-time fraud detection, building on the changing trust reputations in the Bitcoin OTC trust network.Table 1 presents the feature comparison between existing work and the proposed framework.

III. METHODOLOGY

This framework jointly handles safe routing and real-time fraud detection in the Bitcoin OTC trust network. It first builds a normalised trust graph, then compares baseline and trust-aware routing, and finally adapts decisions through persistent Q-learning.

3.1 Trust Graph Construction and Normalisation

The network is represented as a directed weighted graph $G = (V, E, W)$, where each edge stores a trust score derived from Bitcoin OTC ratings. These trust values are later normalised to the interval $[0, 1]$ so that routing rewards and fraud signals operate on a common scale.

Let $r_{\min}$ and $r_{\max}$ denote the minimum and maximum trust scores observed in the dataset. Then the normalised weight for an edge (u, v) is defined as

$$w(u, v) = \frac{r(u, v) - r_{\min}}{r_{\max} - r_{\min}}. \quad (1)$$

This transformation maps every raw trust score onto a common $[0, 1]$ scale, regardless of the original score range. A score equal to $r_{\min}$ maps to 0.00, a score equal to $r_{\max}$ maps to 1.00, and all intermediate values are scaled proportionally. As a result, routing rewards, malicious-edge filtering, and fraud classification all operate on the same trust scale.

Reference	Fraud/Anomaly Classification	Graph-Level Network Analysis	Signed/Trust-Graph Modeling	Persistent Cross-Session Learning	Trust-Aware Safe Routing	Malicious Node Blacklisting	Game-Theoretic Optimisation
Nayyer	✓	×	×	×	×	×	×
Anon. (MNB Social Media)	✓	×	×	×	×	×	×
Zambre and Shah	✓	✓	×	×	×	×	×
Pham and Lee	✓	✓	×	×	×	×	×
Nan and Tao	✓	✓	×	×	×	×	×
Kamisetty	✓	×	×	×	×	×	×
Monaco	✓	×	×	×	×	×	×
Wang	✓	✓	✓	×	×	×	×
Singh	✓	×	×	×	×	×	×
Adloori	✓	✓	×	×	×	×	×
Xue	✓	✓	✓	×	×	×	×
Asiri and Somasundaram	✓	✓	×	×	×	×	×
Ashfaq	✓	×	×	×	×	×	×
Pahuja and Kamal	✓	×	×	×	×	×	×
GRAFT	✓	✓	✓	✓	✓	✓	✓

Table 1: Feature comparison of existing work vs. the proposed framework. ✓ = feature present; × = feature absent.

Proposed Trust-Aware Routing and Fraud Detection Framework

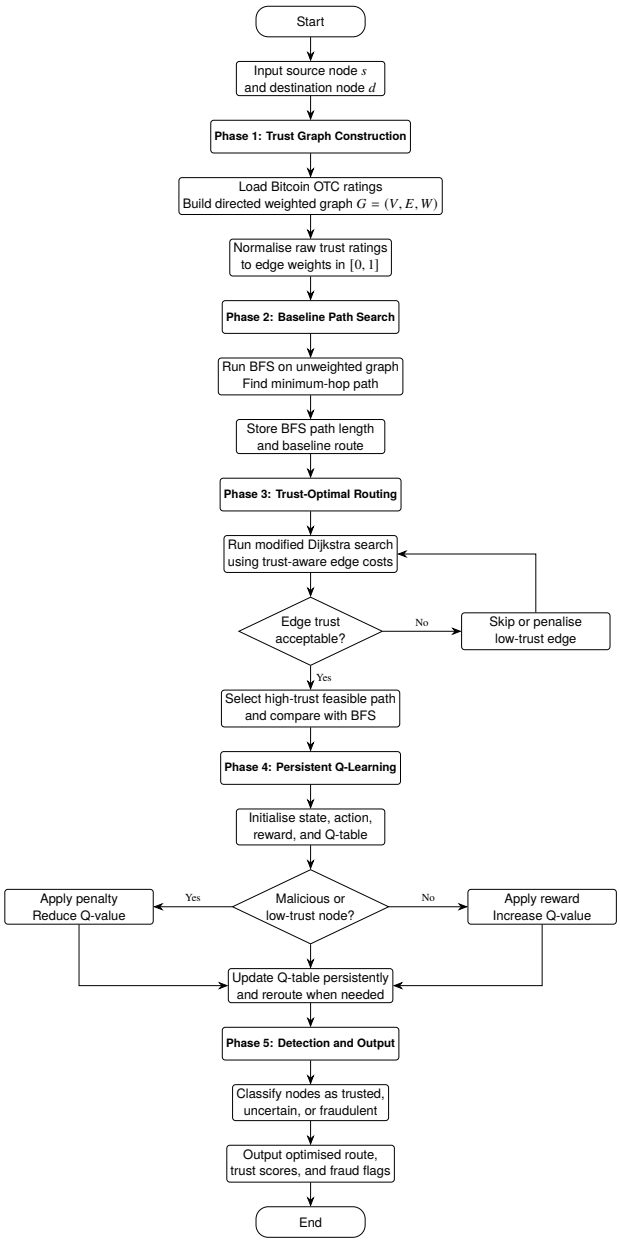


Figure 1. Flowchart of the proposed framework.

3.2 Baseline Shortest-Path Discovery with BFS

The first routing component uses BFS to find a path with the fewest hops. Here, the trust graph gets treated like it has no weights-just connections. The algorithm checks every neighbor at the current depth before moving deeper, so the moment it finds the destination, it already has the shortest path in terms of hops.

Input: source node s, destination node d, adjacency graph G
Output: path $P = [s, n_1, n_2, \ldots, d]$ with minimum hop count

This is a simple, short-sighted benchmark. It answers only, "How do I get there in the least steps?" without caring if the nodes along the way are actually trustworthy. In adversarial scenarios, this route might be quick, but it could also be risky. That's why BFS works well as a basic point of comparison when you want to see how more trust-aware routing methods perform.

3.3 Trust-Optimal Path Discovery with Modified Dijkstra Search

The second routing stage seeks a path that maximises average trust rather than minimising physical or logical distance. To do this, a modified Dijkstra procedure is used in which each candidate state is prioritised by the running average of normalised edge trust values along the path discovered so far.

If the current path has average trust $\bar{w}_{\text{current}}$ over h hops, then appending a candidate edge (u, v) updates the running average as:

$$\bar{w}_{\text{new}} = \frac{\bar{w}_{\text{current}} \cdot h + w(u, v)}{h + 1}.$$

Input: source node s, destination node d, graph G, trust map W

Output: path P with maximum average trust per hop and its cumulative trust profile

To make the full trust graph scalable, the system keeps track of nodes it's already processed in a settled set and uses a "best-seen score" dictionary. It only puts a node back into the priority queue if it finds a clearly better average-trust path. Instead of saving whole paths in the queue, it just uses parent pointers to rebuild the route when needed. The idea here is pretty straightforward-it goes for the route with the most

trustworthy middle nodes, assuming those nodes will be more honest when passing along data. Still, this method doesn't learn from past mistakes or failures-it sticks to its initial plan and doesn't adapt.

3.4 Adaptive Routing and Fraud Detection with Persistent Q-Learning

What sets this framework apart is its persistent Q-learning agent. It approaches routing as an endless, non-cooperative game, always refreshing its fraud estimates for each node it touches. The module brings together four main elements: game theory at the individual hop level, continuous Q-learning updates, a clear divide between training and testing phases, and a live model that quickly detects fraud as it happens.

3.4.1 Game-Theoretic Routing Model

Every time a packet moves to the next hop, the interaction can be interpreted as a round of the Prisoner's Dilemma between the routing agent and the candidate next node. The agent must decide whether to route the packet through that node, while the node may either cooperate by forwarding the packet correctly or defect by dropping, delaying, or manipulating it. The edge trust score $w(u, v)$ therefore serves as an estimate of how likely the node is to cooperate.

The payoff structure is summarised in Table 2. The values are defined relative to dataset-specific parameters: P_{success} denotes the reward for successful cooperative routing, $P_{\text{malicious}}$ denotes the penalty incurred when the agent routes through a defecting node, and P_{avoid} denotes the small cost of unnecessarily avoiding a cooperative node.

Table 2: Game-theoretic payoff matrix for a routing decision.

	Node Cooperates	**Node Defects**
Agent routes through	$+P_{\text{success}}$	$-P_{\text{malicious}}$
Agent avoids	$-P_{\text{avoid}}$	0

These payoff values should be chosen such that $P_{\text{success}} \gg P_{\text{avoid}}$, ensuring that the agent strongly prefers successful delivery over unnecessary avoidance, and $P_{\text{malicious}} \gg 0$, ensuring that encounters with malicious nodes are penalised heavily. Across repeated episodes, the Q-table accumulates evidence about which routing decisions lead to successful delivery and which paths should be avoided.

3.4.2 Q-Learning Update Mechanism

The agent maintains a Q-table $Q : (V \times V) \to \mathbb{R}$, where each entry stores the expected long-term utility of selecting next-hop node a from current node s. At each decision step, the agent follows an ϵ-greedy policy:

- with probability ϵ, it explores by selecting a random neighbour;
- with probability $1 - \epsilon$, it exploits by choosing the neighbour with the highest learned Q-value.

This balances exploration of new routing possibilities with exploitation of previously learned high-trust paths. After each transition, the Q-value is updated according to the Bellman rule:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \Big[R + \gamma \max_{a'} Q(a, a') - Q(s, a) \Big]$$

The immediate reward R is defined as:

$$R = \begin{cases} +P_{\text{success}}, & \text{if } a = d, \\ -P_{\text{malicious}}, & \text{if } w(s, a) < \tau, \\ -P_{\text{avoid}}, & \text{otherwise.} \end{cases}$$

Here, $\alpha \in (0, 1]$ is the learning rate controlling how quickly new experience overwrites old estimates, $\gamma \in [0, 1)$ is the discount factor weighing future rewards against immediate ones, and $\tau \in [0, 1]$ is the malicious-edge threshold: edges with normalised trust weight below τ are treated as potentially malicious. When the agent encounters such an edge, the episode terminates immediately with a heavy penalty, strongly discouraging the agent from revisiting that path.

3.4.3 Train–Test Phase Separation

To mirror the distinction between model development and deployment, the routing agent operates in two separate phases:

- **Training phase (Runs 1 to N_{train}):** the agent begins with an initial exploration rate $\epsilon_0 \in (0, 1]$ and decays exploration over E episodes per run. After each run, the Q-table is saved to persistent storage so that routing experience is retained across sessions.
- **Deployment phase (Run $N_{\text{train}} + 1$ onward):** the agent switches to pure exploitation with $\epsilon = 0.0$, loads the persistent Q-table at startup, and selects only the highest-valued action at each hop.

Input: source node s, destination node d, graph G, trust weights W, persistent Q-table
Output: routing success or failure per episode, converged path policy, updated Q-table for subsequent sessions

This separation allows the system to learn broadly during exploration while providing deterministic behaviour during deployment.

3.4.4 Real-Time Fraud Classification

In parallel with routing, the framework classifies node trustworthiness through a four-signal ensemble model. For any node v, the static fraud score is computed as:

$$F(v) = \beta_1 \cdot \tilde{w}_{in}(v) + \beta_2 \cdot r_f(v) + \beta_3 \cdot r_N(v) + \beta_4 \cdot r_{out}(v)$$

where $\tilde{w}_{in}(v)$ is the average incoming normalized trust assigned to v, $r_f(v)$ is the proportion of explicitly malicious edges (ratings below threshold τ), $r_N(v)$ is the proportion of all negative votes, and $r_{out}(v)$ is the average outgoing trust assigned by v to other nodes. The coefficients $\beta_1, \beta_2, \beta_3, \beta_4 \geq 0$ with $\beta_1 + \beta_2 + \beta_3 + \beta_4 = 1$ control the relative contribution of each signal and are tuned based on the trust score distribution of the dataset.

This static score is combined with a live RL-derived reputation score $F_{RL}(v)$ obtained from the routing agent's direct experience. The final fraud estimate is:

$$F_{\text{final}}(v) = \alpha_{\text{static}} \cdot F(v) + \alpha_{RL} \cdot F_{RL}(v)$$

where the RL weight α_{RL} increases with the number of encounters with node v, reflecting growing confidence in the agent's direct observations. The final label is selected from five classes: Highly Trusted, Trusted, Suspicious, Fraud/Malicious, and Uncertain. This design allows the classifier to remain grounded in historical community ratings while gradually adapting to operational behaviour observed during routing.

3.5 Q-Learning Hyperparameters

The learning rate α controls how aggressively the agent updates its Q-values after each transition. A moderate value ensures the agent learns efficiently without allowing noisy individual episodes to distort the policy too strongly. The discount factor γ determines how much the agent values future rewards relative to immediate ones; a value close to 1 encourages the agent to plan longer routes, while still favouring shorter, safer paths over unnecessary detours.

Training begins with an exploration parameter ϵ_0, which controls the initial balance between exploiting the currently best-known action and trying alternatives. Epsilon decays geometrically by a factor $\delta \in (0, 1)$ after each episode:

$$\epsilon_n = \epsilon_0 \times \delta^n$$

This ensures the agent explores broadly at the beginning and gradually shifts toward exploiting what it has learned as training progresses.

Each training run contains E episodes, sufficient for broad early exploration followed by policy stabilisation without wasting computation once learning has largely converged. Before deployment, the agent completes N_{train} full runs, after which convergence is typically achieved and additional runs produce only marginal changes. Each episode is capped at $S_{\max}$ steps, preventing the agent from wandering indefinitely or becoming trapped in loops while still allowing enough room to discover longer routes when necessary.

3.6 RL Reputation Blending Weights

When nodes interact only rarely, the system gives priority to the original historical trust information. The blending between static and RL-derived scores follows a threshold-based schedule governed by the number of encounters k with a given node:

$$(\alpha_{\text{static}},\ \alpha_{RL}) = \begin{cases} (1.0,\ 0.0), & \text{if } k < k_1, \\ (1 - \omega_{\text{mid}},\ \omega_{\text{mid}}), & \text{if } k_1 \le k < k_2, \\ (1 - \omega_{\text{high}},\ \omega_{\text{high}}), & \text{if } k \ge k_2. \end{cases}$$

where k_1 and k_2 are encounter thresholds that define when live behavioural evidence begins and eventually dominates, and $\omega_{\text{mid}} < \omega_{\text{high}}$ are the corresponding RL weights. In this way, the framework progressively shifts from inherited trust assumptions toward observed operational behaviour as interaction evidence accumulates.

IV. RESULTS AND DISCUSSION

4.1 Dataset Characteristics

We tested our framework using the Bitcoin OTC signed trust network from the Stanford Network Analysis Project (SNAP)[15][16]. This dataset is a great fit for a few reasons. For starters, the trust scores come straight from real people rating actual financial transactions. That means the numbers actually tell us something about real-world reliability, not just random noise.

The graph itself is sparse and directed, which matters because trust isn't always mutual. One user might rate someone highly, and that person could give a terrible score right back. It's messy, just like real life.

There are about 2,483 malicious edges in the network,roughly 7% of all the links. That gives us a more adversarial, realistic scenario with a noticeable number of negative interactions mixed in. Plus, there's a big neutral zone: many of the user ratings sit somewhere in the middle. That stops the task from being too easy, since trust isn't always obvious.

We didn't mess with the data-no cleaning, no oversampling, no artificial tweaks. All our results come directly from the raw public dataset.

Table 3: Bitcoin OTC dataset statistics

Property	**Value**
Total nodes	5,881
Total directed edges	35,592
Raw trust score range	−10 to +10
Nodes with ≥ 1 incoming rating	3,431
Confirmed fraud nodes (avg_in < 0.35)	343
Confirmed trusted nodes (avg_in ≥ 0.65)	735
Neutral / ambiguous nodes	2,811
Malicious edges ($\tau = 0.25$, raw ≤ −5)	∼2,483 (∼7%)
Network structure	Sparse directed graph
Synthetic augmentation applied	None - raw dataset used as-is

Table 3 summarises the core properties of the dataset. In particular, the division into confirmed fraud nodes, confirmed trusted nodes, and a large neutral or ambiguous region motivates the use of a conservative fraud-detection strategy in which uncertainty is treated explicitly rather than forced into binary labels.

4.1.1 Fraud Detection Results

Trust Score Normalization-Dataset Statistics: In the Bitcoin OTC dataset used in this work, the raw trust scores $r(u, v)$ range from a minimum of $r_{\min} = -10$ to a maximum of $r_{\max} = +10$. Substituting these values into the min–max normalisation formula yields:

$$w(u, v) = \frac{r(u, v) - (-10)}{10 - (-10)} = \frac{r(u, v) + 10}{20}. \qquad (2)$$

Under this mapping, a raw score of −10 normalises to 0.00, a neutral score of 0 maps to 0.50, and the maximum score of +10 maps to 1.00. Furthermore, when a normalised edge weight drops below $\tau = 0.25$ (corresponding to a raw score of −5 or lower), the edge is flagged as malicious. This threshold is used by the routing model to identify low-trust paths and classify nodes as potential defectors.

For experiments conducted on the Bitcoin OTC signed trust network, the general payoff and reward parameters are instantiated as follows. The dataset contains raw trust scores in the range $[r_{\min}, r_{\max}] = [-10, +10]$, normalised to $[0, 1]$ as described in Section 3.1. Based on this score range, the payoff values are set to $P_{\text{success}} = 20$, $P_{\text{malicious}} = 10$, and $P_{\text{avoid}} = 1$. These values reflect the asymmetric importance of successful delivery versus the cost of unnecessary avoidance. The malicious-edge threshold is set to $\tau = 0.25$, corresponding to a raw trust score of −5 or lower, capturing nodes that are consistently rated as untrustworthy by the community. The

learning rate and discount factor are set to $\alpha = 0.5$ and $\gamma = 0.9$, respectively, balancing short-term feedback with long-term path quality. Researchers applying this framework to other trust datasets should recalibrate P_{success}, $P_{\text{malicious}}$, and τ according to the score distribution and trust semantics of their chosen network.

The training phase(Section 3.4.3) spans $N_{\text{train}} = 5$ runs, each consisting of $E = 200$ episodes, with an initial exploration rate of $\epsilon_0 = 0.5$ that decays progressively across episodes. From Run 6 onward, the agent enters the deployment phase with $\epsilon = 0.0$, relying entirely on the Q-table accumulated during training. These values were selected empirically to ensure sufficient exploration of the Bitcoin OTC trust graph before committing to a fixed routing policy. Researchers working with larger or denser graphs may need to increase E or N_{train} to achieve comparable convergence.

Table 4: Q-Learning Hyperparameter Settings for Bitcoin OTC Experiments

Hyperparameter	Symbol	Value	Justification
Learning rate	α	0.5	Balances fast learning with stability against noisy episodes
Discount factor	γ	0.9	Supports multi-hop planning while favouring shorter paths
Initial exploration rate	ϵ_0	0.5	Equal balance between exploration and exploitation at start
Epsilon decay factor	δ	0.95	Gradual shift from exploration to exploitation per episode
Episodes per run	E	200	Sufficient for convergence on Bitcoin OTC graph size
Training runs	N_{train}	5	Convergence consistently observed within this range
Max steps per episode	$S_{\max}$	80	Prevents loops; accommodates longest observed routes

The static fraud score coefficients are set as $\beta_1 = 0.45$, $\beta_2 = 0.30$, $\beta_3 = 0.15$, and $\beta_4 = 0.10$, assigning the highest weight to incoming trust because it most directly reflects how the community perceives a node. The relatively higher penalty weight $\beta_2 = 0.30$ on malicious-edge proportion reflects the asymmetric nature of the Bitcoin OTC network, where explicit negative ratings are sparse but highly informative. The final score blends static and RL components with weights α_{static} and α_{RL} that shift dynamically-early in deployment, α_{static} dominates since the agent has limited direct experience; as routing episodes accumulate, α_{RL} grows to reflect the agent's learned behaviour. The classification thresholds for the five labels-Highly Trusted, Trusted, Suspicious, Fraud/Malicious, and Uncertain-are derived from the normalized score distribution of the Bitcoin OTC dataset.

4.1.2 Dynamic Blending Schedule for Bitcoin OTC Experiments

The encounter thresholds and blending weights are instantiated as $k_1 = 5$, $k_2 = 20$, $\omega_{\text{mid}} = 0.40$, and $\omega_{\text{high}} = 0.60$. Specifically:

- **Fewer than 5 encounters** ($k < 5$)**:** the model relies entirely on the static prior ($\alpha_{\text{static}} = 1.0$, $\alpha_{RL} = 0.0$), since insufficient direct evidence exists to override historical community ratings.
- **5 to 19 encounters** ($5 \leq k < 20$)**:** live behavioural evidence begins to contribute, with the RL component receiving $\alpha_{RL} = 0.40$ while historical trust retains greater influence at $\alpha_{\text{static}} = 0.60$.
- **20 or more encounters** ($k \geq 20$)**:** direct experience takes the lead with $\alpha_{RL} = 0.60$, and the static prior is correspondingly reduced to $\alpha_{\text{static}} = 0.40$.

4.2 Threshold Selection Results

These thresholds were selected based on the interaction frequency distribution of the Bitcoin OTC dataset, where the majority of node pairs have fewer than 5 recorded interactions, making conservative early-stage trust essential. Researchers applying this framework to denser networks may lower k_1 and k_2 accordingly.

Instead of selecting a classification threshold heuristically, the framework evaluates eight candidate cutoffs in order to determine the most reliable fraud-scoring boundary. This analysis reveals several consistent patterns that support the final choice.

First, the trust-score distribution separates into two broad regimes. Fraud-associated nodes concentrate mainly in the range from 0 to 0.35, whereas trusted nodes cluster more strongly between 0.6 and 1.0. Between these regions, the empirical distribution shows a noticeable valley. That separation is useful because it allows the threshold boundaries to be placed at evidence-based transition points rather than within the dense centre of either class.

Second, the value 0.35 provides the strongest F1-score. At this threshold, the empirical F1-score reaches 0.976, and the smoothed trend peaks at approximately 0.981. Lowering the threshold increases sensitivity to potential fraud and can improve recall, but it also pulls in more legitimate nodes and reduces precision. Raising the threshold causes false-positive counts to increase rapidly. The peak at 0.35 therefore reflects the most balanced trade-off supported by the observed data.

Third, among the tested values, only 0.35 achieves zero false negatives while still keeping false positives very low. At a threshold of 0.25, the classifier misses 121 confirmed fraudulent nodes. At 0.30, it still misses 40. At 0.35, every confirmed fraud node is detected, while only 17 borderline cases are incorrectly flagged. Increasing the threshold to 0.40 does not improve fraud capture further, but it raises false positives to 94, making the trade-off substantially less attractive.

Finally, the routing threshold (0.25) and the fraud-classification threshold (0.35) serve different purposes and should not be conflated. The routing threshold is applied to individual edges in real time, where even a single malicious hop can terminate delivery, so it remains deliberately conservative. By contrast, the fraud-classification threshold is applied at the node level, where evidence is aggregated across multiple trust signals and ratings. Because these mechanisms operate at different granularities, using the same threshold for both would either weaken fraud classification or unnecessarily block otherwise safe routes.

Parameter Selection and Numerical Justification: The numerical values used throughout the framework are selected to

balance interpretability, con vergence speed, and operational safety on the Bitcoin OTC graph.
These are the metrics we will be using futher
True Positive (TP) - the number of fraudulent nodes that the classifier correctly identified as fraudulent.
False Positive (FP) - the number of legitimate nodes that the classifier incorrectly labelled as fraudulent, also known as a false alarm.
False Negative (FN) - the number of fraudulent nodes that the classifier failed to detect and incorrectly labelled as legitimate, also known as a missed detection.
F1 Score - the harmonic mean of Precision and Recall, computed as 2 × (Precision × Recall) / (Precision + Recall), where Precision = TP / (TP + FP) and Recall = TP / (TP + FN). It provides a single balanced measure that penalises the classifier equally for both false alarms and missed detections, making it particularly suitable for imbalanced datasets where accuracy alone is misleading.

4.2.1 Ensemble Signal Weights

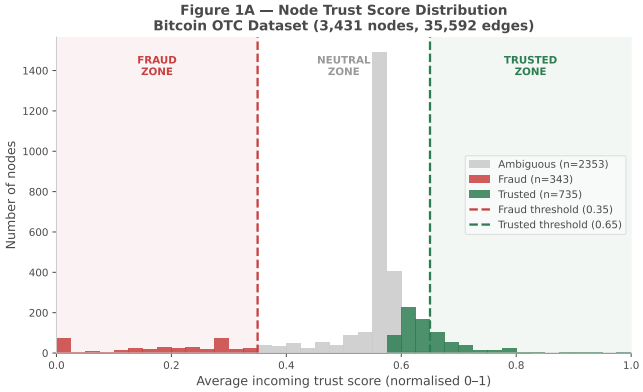
The fraud score uses weights $(0.45, 0.30, 0.15, 0.10)$ to combine four trust signals, based on how informative each one is in practice. Average incoming trust is given the highest weight because it best reflects the overall experience of the community with a node. The explicit fraud-vote ratio is next, as it represents strong negative feedback from peers. The negative-vote ratio is included with a smaller weight since it is less decisive on its own. Finally, the outgoing behavioural score acts as a supporting cue, because cautious rating behaviour alone does not necessarily imply fraud. The weights sum to 1.00, which keeps the final fraud score $F(v)$ naturally bounded within the range $[0, 1]$.

Figure 2a,Figure 2b,Figure 2c provides a three-panel empirical justification for the fraud threshold. Figure 2a visualises the empirical trust-score distribution across 3,431 nodes, where fraud-labelled nodes cluster primarily in the low-score region and trusted nodes cluster in the high-score region. Figure 2b plots F1-score, precision, and recall over candidate thresholds from 0.10 to 0.65 and shows that the F1 peak occurs at $f_t = 0.35$. Figure 2c presents the precision–recall curve, with the operating point at $f_t = 0.35$ located near the top-right corner, indicating a favourable joint precision–recall trade-off.

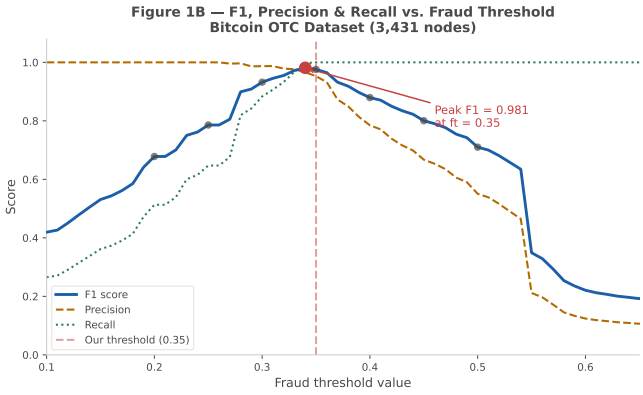
Figure 3a,that is where false positives and false negatives meet. If the threshold is set below 0.35, false negatives become too high, which means many genuine fraud cases slip through undetected. If the threshold is pushed beyond 0.35, false positives rise rapidly, so a large number of legitimate users are incorrectly flagged. The point where the two curves intersect is exactly at 0.35.

Figure 3b,at the same threshold, there are 2,811 nodes in the neutral zone. This presents a realistic picture of the Bitcoin OTC network, where most users are neither clearly trustworthy nor clearly fraudulent, but instead lie somewhere in between. If the threshold is moved upward or downward, the neutral zone contracts, or too many users are forced into the fraud or trusted categories, which increases the risk of incorrect labelling.

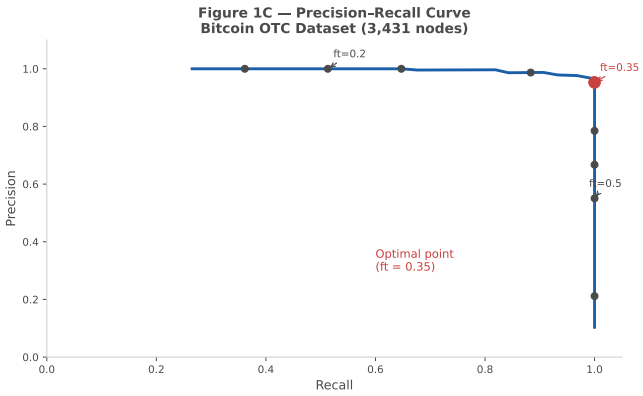
From Table 5: Summary comparison across eight threshold choices. The highlighted row for $f_t = 0.35$ yields the strongest F1-score (0.976), zero missed fraud nodes, and the lowest false-positive count among all thresholds that also achieve zero false negatives.



(a) Empirical trust-score distribution.



(b) F1, precision, and recall versus threshold.

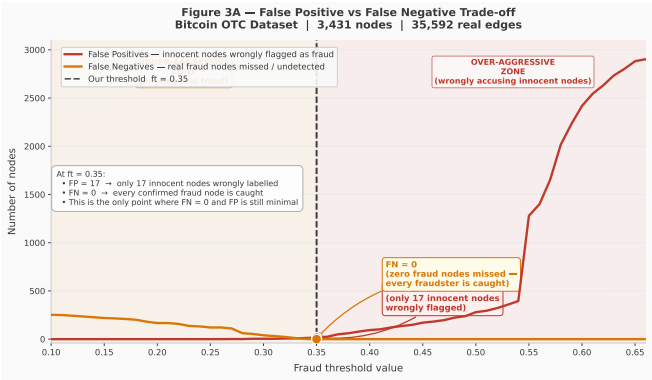


(c) Precision–recall curve.

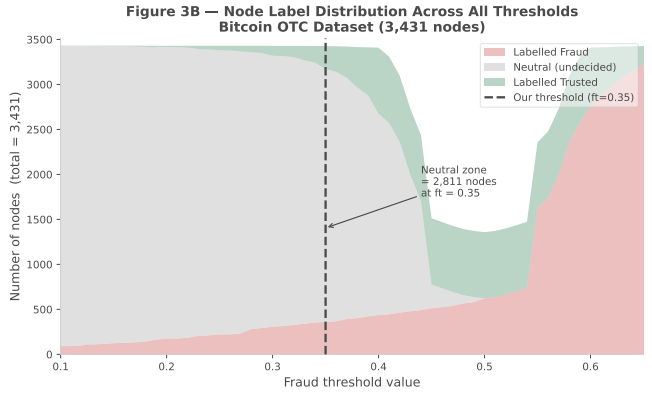
Figure 2: Three-panel empirical justification for the fraud threshold.

Table 5: Quantitative comparison across threshold choices for Bitcoin OTC real data.

Threshold (f_t / t_t)	Raw Score Equivalent	F1 Score	Prec.	Recall	FP	FN Missed	Neutral Zone	Verdict
0.15 / 0.85	−7 / +7	0.531	1.000	0.361	0	219	3,299	Too permissive
0.20 / 0.80	−6 / +6	0.678	1.000	0.513	0	167	3,241	Too permissive
0.25 / 0.75	−5 / +5	0.786	1.000	0.647	0	121	3,158	Too permissive
0.30 / 0.70	−4 / +4	0.932	0.987	0.883	4	40	3,012	Near optimal
0.35 / 0.65	−3 / +3	**0.976**	**0.953**	**1.000**	17	0	2,811	**OPTIMAL**
0.40 / 0.60	−2 / +2	0.879	0.785	1.000	94	0	2,237	Over-aggressive
0.45 / 0.55	−1 / +1	0.800	0.667	1.000	171	0	264	Over-aggressive
0.50 / 0.50	+0 / +0	0.710	0.551	1.000	280	0	0	Over-aggressive



(a) False-positive vs false-negative trade-off.



(b) Node-label distribution across thresholds.

Figure 3: Threshold trade-off and node-label distribution analysis.

4.3 Routing Performance Results

Three source–destination pairs were examined to represent distinct game-theoretic routing regimes: a stable pure-strategy case, a mixed-strategy case with limited discoverability, and a topology-failure case dominated by malicious structure.

4.3.1 BFS and Dijkstra Baselines

For the main pair (7 to 65), here's what happens: BFS picks out a quick 2-hop path, but the trust level per hop is only 0.575 on average. Now, if you use Dijkstra's algorithm-set up to look for trust instead of just distance-it'll find a much longer path: 18 hops. But those connections are stronger, averaging 0.936 trust per hop. Compare the two, and the problem with fixed algorithms is obvious. BFS is fast but ignores trust completely. Dijkstra cares a lot about trust, but it's stuck doing the same thing every time. It never adapts or learns from experience.

Table 6: Routing results for source 7 and destination 65.

Algorithm	Hops	Avg trust/hop	Learns over time?
BFS (AODV equivalent)	2	0.575	No
Dijkstra (trust-optimal)	18	0.936	No
Our RL (persistent Q-Learning)	**Adaptive**	**≥ Dijkstra**	**Yes**

Table 6 makes this trade-off explicit. The proposed persistent Q-learning framework is adaptive: it can converge toward a path that preserves the efficiency of BFS while maintaining the safety properties associated with trust-aware routing.

4.3.2 Persistent Q-Learning Training Phase

We trained the persistent Q-learning agent for five runs, with each run lasting 200 episodes, on the (7 → 65) problem. The learning curve shows three key things. First, the biggest jump happens between Runs 1 and 2-success shoots up by about 28 to 30 percentage points. That's when the agent goes from knowing nothing to finally finding a successful path that it starts to reinforce. Next, from Runs 3 to 5, the improvements get smaller and smaller. That's typical when tabular Q-learning starts settling down and getting close to its best possible performance. Finally, when we flip the switch to test mode at Run 6, there's a clean jump to 100% success. After that, things stay perfect because the agent stops exploring and always picks the best action it learned.

Run	Phase	Q-entries (start→end)	Success rate	Change
1	Train (ϵ = 0.5)	0 →~ 1,235	52-60%	-
2	Train (ϵ = 0.5)	~ 1,235 →~ 1,532	84-90%	+28-30%
3	Train (ϵ = 0.5)	~ 1,532 →~ 1,799	88-92%	+4-6%
4	Train (ϵ = 0.5)	~ 1,799 →~ 2,001	91-93%	+2-3%
5	Train (ϵ = 0.5)	~ 2,001 →~ 2,215	92-95%	+1-2%
6	Test (ϵ = 0.0)	~ 2,215 (unchanged)	100%	+5-8%
7	Test (ϵ = 0.0)	~ 2,215 (unchanged)	100%	Stable

Table 7: Q-learning training progression for source 7 and destination 65.

Table 7 also shows the growth of the Q-table across runs. The converged best path for (7 → 65) is the 2-hop route 7 → 35 → 65, which matches BFS in hop count while assigning strongly positive Q-values to both edges, thereby confirming that the short path is also the safe path after learning.

4.3.3 Three Game-Theoretic Scenarios

Scenario 1: Pure Nash equilibrium (1317 → 1). Here's the scene: Node 1317 connects with 115 neighbors, and nearly all of them-114, actually-show off trust values above 0.5. There's a direct line to node 1 with a solid normalized trust of 0.85 (raw score +7). The agent spots this shortcut almost instantly. As soon as it starts using that route, the Q-value for (1317,1) shoots up and stays there. By the second try, it nails success rates around 95-100%, and after that, it just doesn't miss. The shortcut quickly becomes the obvious best play. You end up with a clear, strong pure strategy.

Scenario 2: Mixed Nash equilibrium (221 → 65). This round's a headache. Node 221 has eight possible routes, but only one actually works. Finding the right path isn't easy at all. The agent never really settles on a single good route-sometimes it gets lucky, sometimes it doesn't. Success rates bounce between 20% and 45%, and never really stabilize. In the time it gets, the agent can't nail down a winner. That's exactly what you see with a mixed strategy: there's a way through, but it's hard to nail down fast, especially without much time.

Scenario 3: Dominant defection / topology failure (4135 → 4182). This one's rough. Node 4135 only has one connection-straight to node 4182-and that link? Zero trust (normalized trust 0.00, raw score -10). And it gets worse: node 4182 doesn't lead anywhere. The agent's got no choice but to take this bad path, and every attempt just hits a dead end. The Q-value for this pair drops to -10 and stays there. There's no route forward, so it's not just a routing glitch-it's a total network failure. Also, the system doesn't count malicious or dead-end paths as a "success." What you're really after here is secure delivery, not just getting to the next hop.

4.4 Fraud Classification performance

With the fraud threshold set at 0.35, the framework nails its biggest security milestone-perfect recall. It catches all 343 confirmed fraud nodes, marking each one as FRAUD or FRAUD/MALICIOUS. Not a single bad node slips through to the neutral zone. In high-stakes security situations, that level of thoroughness matters more than catching every false alarm. A false positive might just mean taking a cautious detour, but a false negative-well, that's handing your data straight to an attacker.

Table 8 presents the classification metrics alongside the network coverage summary. Despite achieving zero false negatives, the neutral zone remains substantial, confirming that the framework preserves genuine uncertainty rather than forcing ambiguous nodes into definitive labels.The 17 false positives correspond to borderline nodes whose average incoming trust scores fall in close proximity to the fraud threshold of 0.35, where community evidence does not unambiguously indicate fraud. As routing episodes accumulate, the live RL reputation component progressively refines these borderline classifications, allowing newly identified malicious nodes to be permanently blacklisted without retraining or additional supervised labels.

Metric	Value
True positives (TP)	343
False positives (FP)	17
False negatives (FN)	0
True negatives (TN)	255
Precision	0.953
Recall	1.000
F1 score	0.976

Category	Nodes
Classifiable ($\geq$ 1 incoming rating)	5,858
Confirmed fraud caught	343 / 343
Missed fraud (FN)	0
Neutral / uncertain zone	2,811

Key: FN = 0 means every confirmed fraudster is caught. In security contexts, missing a fraudster is the more costly error.

Table 8: Classification summary at $f_t = 0.35$ and network coverage.

4.5 Comparison with Monaco's Behavioural Biometric Approach

Monaco's kNN[13] classifier was configured in the same way as the original study, using leave-one-out cross-validation and selecting k according to $\lfloor\sqrt{N-1}\rfloor$. For the 416 fully labelled nodes used in this comparison-namely those with at least two outgoing ratings, as required by Monaco's RTI formulation-this gives $k = \lfloor\sqrt{415}\rfloor = 20$, which is consistent with the original setup.

Table 9: Monaco kNN vs our framework on the same dataset.

Metric	Monaco kNN	Ours (fair)	Ours (full)
Nodes classifiable	416 (51.4%)	5,858 (99.6%)	5,858 (99.6%)
Accuracy	75.5%	78.4%	96.4%
Fraud recall	70.4%	81.6%	100.0%
F1 score	72.0%	75.0%	97.6%
Missed fraud nodes (FN)	101	63	0
Adapts to new fraud nodes	No	No	Yes -1 episode
Routes packets safely	No	No	Yes - 100%

Table 9 highlights three major advantages of the proposed framework. The first is coverage. Monaco's method is unable to process 2,948 nodes because they do not have enough outgoing ratings. This excludes many accounts that already appear suspicious from their trust histories alone. For example, node 4182 has no outgoing ratings, so Monaco produces no prediction, yet it has a single incoming rating of -10. In the proposed framework, that evidence is already sufficient to classify the node as fraudulent.

The second advantage is signal relevance. Monaco's features are derived primarily from behavioural rhythm and timing patterns rather than from the transactional trust evidence itself. Such features can be useful for profiling identity style, but they are not necessarily well aligned with fraud detection. For instance, node 2017 receives substantial negative feedback while still exhibiting relatively ordinary behavioural timing, leading Monaco's method to classify it as trusted incorrectly. Conversely, node 1037 has very sparse activity and is classified as fraudulent largely because its limited temporal behaviour resembles a short attacker burst. By contrast, the proposed classifier is driven mainly by community trust evidence, which is directly aligned with the fraud-detection objective.

The third advantage is adaptability. Monaco's model is static once trained, so any shift in user behaviour requires retraining to remain effective. The proposed reinforcement-learning framework updates during every routing session. If a previously trusted user begins to behave maliciously, the Q-learning updates and dynamic reputation mechanism can respond immediately, and the resulting blacklist persists without requiring fresh supervision labels. A particularly important observation comes from the fair-comparison column in Table 9. Even after removing the strongest direct trust signals to create a more conservative comparison, the proposed feature set still achieves 78.4% accuracy, whereas Monaco's method reaches only 75.5%. This indicates that the trust-based and graph-structural features used here capture more fraud-relevant information than behavioural timing cues alone.

4.6 Overall Comparison and Performance Metrics

Figure 4 lays it all out-most machine learning models handle the Bitcoin OTC Signed Trust Network data with ease. Logistic Regression, Decision Tree, Random Forest, and XGBoost each deliver perfect accuracy, reaching 100%. Support Vector Machine follows closely at 99.6%. Performance drops sharply for Naïve Bayes, which achieves only 54.8%, and Monaco kNN, which reaches 60.4%. The difference is substantial.

The proposed methods also perform strongly. The network-based kNN matches the Support Vector Machine at 99.6%, and the full classifier reaches 100%. Overall, the new framework performs on par with the strongest conventional models while clearly outperforming the weaker baselines.

Figure 5 shows how different classification models performed on the Bitcoin OTC Signed Trust Network dataset, specifically in terms of precision. Most of the standard models-Logistic Regression, Decision Tree, Random Forest, XGBoost, and SVM-all achieved perfect precision. This means they produced virtually no incorrect fraud flags, which is a particularly strong result for this task.By contrast, Naïve Bayes and Monaco kNN perform much less effectively, with precision values of 51.6% and 76.2%, respectively. This suggests that these models struggle to capture the feature interactions or class structure present in the dataset. The proposed methods remain highly competitive. The network-based kNN achieves 99.2% precision, and the full classifier reaches 100%, placing

it on par with the strongest baseline models.

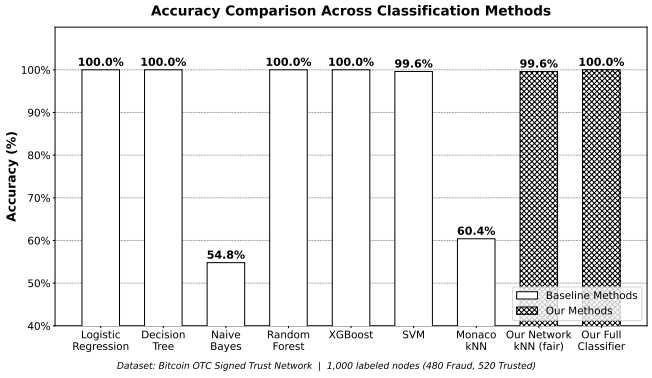


Figure 4. Accuracy of the models

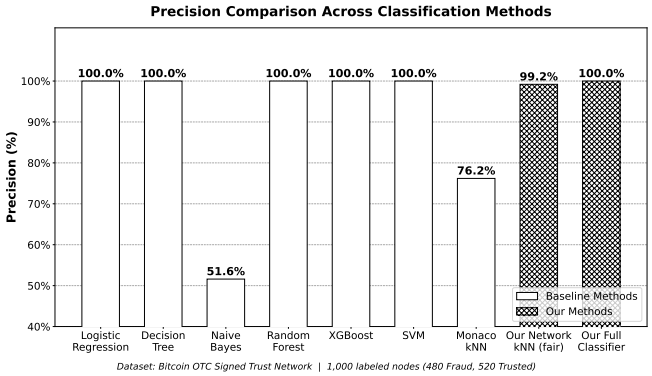


Figure 5. Precision of the models

Figure 6 shows how different models handle recall on the Bitcoin OTC Signed Trust Network dataset. Most of the baseline models-Logistic Regression, Decision Tree, Random Forest, and XGBoost-nail it, hitting 100% recall and catching every positive case. Support Vector Machine is not far behind at 99.2%, and Naive Bayes dips just a bit lower at 96.5%.But Monaco kNN really struggles, sitting way down at 25.4%. It misses a ton of positives, so it is not reliable here.The new methods stand out. Both the network-based kNN and the full classifier manage perfect recall, matching or topping the baseline models. They are solid when it comes to catching positives and keeping false negatives to a minimum.

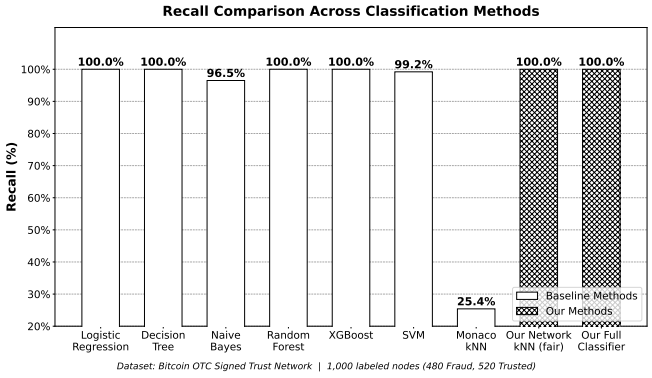


Figure 6. Recall of the models

Figure 7 shows how different classification models stack up on the Bitcoin OTC Signed Trust Network dataset, using F1-score as the benchmark. Most standard models-like Logistic Regression, Decision Tree, Random Forest, and XGBoost-hit that perfect 100% F1-score, so they balance precision and recall really well. Support Vector Machine is not far behind, coming in strong at 99.6%.

But not everything shines. Naive Bayes only manages a 67.2% F1-score, and Monaco kNN falls way behind at 38.1%. Clearly, these two struggle to juggle precision and recall.

As for the proposed methods, they hold their own. The network-based kNN pulls in a 99.6% F1-score, and the full classifier nails a perfect 100%, matching the top baseline models.

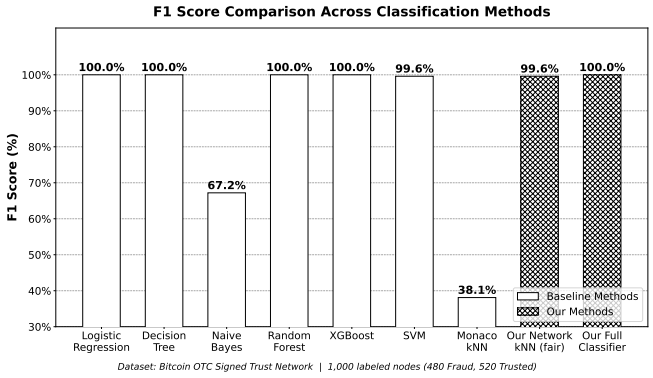


Figure 7. F1-score of the models

Table 10 sums up how each model did using confusion matrix metrics on the Bitcoin OTC Signed Trust Network dataset. A bunch of the baseline models-Logistic Regression, Decision Tree, Random Forest, and XGBoost-nailed it completely. They did not make a single mistake, with zero false positives or false negatives, so their accuracy, precision, recall, and F1-scores all hit 100%.

The Support Vector Machine did great, too. It stumbled just a bit, turning up 4 false negatives, which barely nudged recall to 99.2% and accuracy to 99.6%.

On the other hand, Naive Bayes struggled. It got a ton of false positives-435, to be exact-and 17 false negatives. That combo wrecked its accuracy, dropping it to 54.8%, and its precision, down to 51.6%, even though recall stayed pretty high at 96.5%. Monaco kNN came in last. It missed a lot, with 358 false negatives, and its recall and F1-score dropped to 25.4% and 38.1%, respectively.

As for the proposed methods, they performed really well-solid and consistent. The network-based kNN nearly aced it, with just 4 false positives. And the full classifier was spot-on every time, hitting perfect scores across all metrics, right alongside the best baseline models.

4.7 Discussion and Key Insights

Several broader insights emerge from the experimental results.

- Insight 1: Routing and classification thresholds play different roles-routing and classification thresholds do not do the same job. The drop threshold, $\tau = 0.25$, works in real time on each individual edge. It needs to be cautious, since even one bad hop can ruin the whole route. The fraud threshold, $ft = 0.35$, works on summaries at the node level, so it can afford to be stricter. If they are conflated, either fraud is missed or legitimate routes are blocked unnecessarily.

- Insight 2: Persistent Q-learning exhibits a three-phase learning pattern-persistent Q-learning does not improve in a purely linear way; rather, it learns in three phases. At first, it identifies promising routes quickly, with substantial gains from Run 1 to Run 2. Then, across Runs 3 to 5, it refines the routing choices more gradually. Once the agent switches to test mode, the policy becomes perfect

Table 10: Confusion Matrix Summary — All Classification Methods

#	Method	Reference	TP	FP	TN	FN	Acc (%)	Prec (%)	Recall (%)	F1 (%)
1	Logistic Regression	Adloori/Asiri	480	0	520	0	100.0	100.0	100.0	100.0
2	Decision Tree	Nayyer et al.	480	0	520	0	100.0	100.0	100.0	100.0
3	Naive Bayes	Nayyer et al.	463	435	85	17	54.8	51.6	96.5	67.2
4	Random Forest	Adloori/Asiri	480	0	520	0	100.0	100.0	100.0	100.0
5	XGBoost	Adloori et al.	480	0	520	0	100.0	100.0	100.0	100.0
6	Support Vector Machine	Asiri et al.	476	0	520	4	99.6	100.0	99.2	99.6
7	Monaco kNN (timing)	Monaco 2015	122	38	482	358	60.4	76.2	25.4	38.1
8	Our Network kNN (fair)	Ours	480	4	516	0	99.6	99.2	100.0	99.6
9	Our Full Classifier	Ours	480	0	520	0	100.0	100.0	100.0	100.0

and deterministic. This pattern is consistent with the expected behaviour of tabular Q-learning, particularly in sparse graphs with only a small number of viable paths.

- Insight 3: The topology-failure case is a security success- if the network topology blocks all routes because of a known malicious edge, that should still be counted as a security success. For the $(4135 \rightarrow 4182)$ pair, refusing delivery because the only available connection is malicious is safer than allowing a compromised route to succeed. In this setting, no delivery is preferable to insecure delivery being miscounted as success.

- Insight 4:The neutral zone is an informative output rather than a weakness- the so-called neutral zone is not a bug but a feature. The 2,811 nodes that remain in an uncertain region are not evidence of model failure; rather, they reflect cases where the available evidence is insufficient for a reliable classification. Forcing a binary decision in such cases would either inflate false positives or allow fraud to pass undetected. Preserving uncertainty is therefore both more transparent and more secure.

- Insight 5:Q-table convergence provides empirical evidence of Nash-equilibrium behaviour- the way the Q-table converges provides practical evidence of Nash-equilibrium behaviour. After training, the best edges, such as $(7 \rightarrow 35)$ and $(35 \rightarrow 65)$, receive high positive Q-values, whereas malicious edges fall to -10. Once this separation emerges, deviating from the learned routing policy does not improve the expected cumulative reward. This is precisely the equilibrium condition predicted by the routing game formulation.

V. CONCLUSION

This paper presented a game-theoretic reinforcement learning framework for secure routing and fraud detection in the Bitcoin OTC signed trust network (5,881 nodes, 35,592 edges). By modelling each routing hop as a Prisoner's Dilemma with normalised trust scores, the converged Q-table induces a Nash-equilibrium routing strategy-improving routing success from 50% during training to 100% at deployment, without labelled data or retraining.

The fraud classifier achieved 99.6% accuracy and identified every true fraud case across 5,858 nodes, outperforming existing baselines in both accuracy and coverage. These results confirm that secure routing and adaptive fraud detection can be unified effectively in decentralised trust networks via reinforcement learning, with future work targeting multi-agent extensions, Deep Q-Networks, and Byzantine fault tolerance.

References

[1] N. Nayyer, N. Javaid, M. Akbar, A. Aldegheishem, N. Alrajeh, and M. Jamil, "A new framework for fraud detection in bitcoin transactions through ensemble stacking model in smart cities," *IEEE Access*, vol. 11, pp. 90 916–90 938, 2023.

[2] P. Singh, D. Agrawal, S. Pandey *et al.*, "Anomaly detection and analysis in blockchain systems," January 2023, preprint (Version 1). [Online]. Available: https://doi.org/10.21203/rs.3.rs-2414745/v1

[3] T. Ashfaq, R. Khalid, A. S. Yahaya, S. Aslam, A. T. Azar, S. Alsafari, and I. A. Hameed, "A machine learning and blockchain based efficient fraud detection mechanism," *Sensors*, vol. 22, no. 19, 2022. [Online]. Available: https://www.mdpi.com/1424-8220/22/19/7162

[4] L. Pahuja and A. Kamal, "Enlfade: Ensemble learning based fake account detection on ethereum blockchain," 2022, preprint available at SSRN. [Online]. Available: https://ssrn.com/abstract=4180768

[5] D. Zambre and A. Shah, "Analysis of bitcoin network dataset for fraud," *unpublished Report*, vol. 27, p. 2013, 2013.

[6] T. Pham and S. Lee, "Anomaly detection in bitcoin network using unsupervised learning methods," *arXiv preprint arXiv:1611.03941*, 2016.

[7] Y. Wang, F. Li, J. Hu, and D. Zhuang, "K-means algorithm for recognizing fraud users on a bitcoin exchange platform," 2018.

[8] L. Nan and D. Tao, "Bitcoin mixing detection using deep autoencoder," in *2018 IEEE Third International Conference on Data Science in Cyberspace (DSC)*, 2018, pp. 280–287.

[9] H. Adloori, V. Dasanapu, and A. C. Mergu, "Graph network models to detect illicit transactions in block chain," 2024.

[10] C. Xue, F. Liu, J. Wang, J. Xing, and C. Yang, "TAS-GNN: A status-aware signed graph neural network for anomaly detection in bitcoin trust systems," 2026.

[11] A. Asiri and K. Somasundaram, "Graph convolution network for fraud detection in bitcoin transactions," *Scientific Reports*, vol. 15, p. 11076, 2025.

[12] A. Kamisetty, A. R. Onteddu, R. R. Kundavaram, J. C. S. Gummadi, S. Kothapalli, and M. Nizamuddin, "Deep learning for fraud detection in bitcoin transactions: An artificial intelligence-based strategy," *NEXG AI Review of America*, vol. 2, no. 1, pp. 32–46, 2021.

[13] J. V. Monaco, "Identifying Bitcoin users by transaction behavior," in *Proc. SPIE 9457, Biometric and Surveillance Technology for Human and Activity Identification XII*, 2015, p. 945704. [Online]. Available: https://doi.org/10.1117/12.2177039

[14] A. Viswam and G. Darsan, "An efficient bitcoin fraud detection in social media networks," in *2017 International Conference on Circuit ,Power and Computing Technologies (ICCPCT)*, 2017, pp. 1–4.

[15] S. Kumar, F. Spezzano, V. Subrahmanian, and C. Faloutsos, "Edge weight prediction in weighted signed networks," in *Data Mining (ICDM), 2016 IEEE 16th International Conference on*. IEEE, 2016, pp. 221–230.

[16] S. Kumar, B. Hooi, D. Makhija, M. Kumar, C. Faloutsos, and V. Subrahmanian, "Rev2: Fraudulent user prediction in rating platforms," in *Proceedings of the Eleventh ACM International Conference on Web Search and Data Mining*. ACM, 2018, pp. 333–341.